Assignment 2

FIKRI BIN ABDULLAH | 24005888

Questions

1. Why does BFS ignore the distances stored in the graph even though they are available to it?

   BFS ignores distances because it treats every edge as having equal cost and aims to find path with fewest flights (hops), rather than the shortest total distance in kilometres.

2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?

   Two lines that is changed are the frontier data structure initialization where the queue becomes stack and the removal operation where pop from front becomes pop from top or end. LIFO removal operation where pop from top makes the DFS order-dependant.

3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?

   It returns a sub-optimal path. The node's shortest path is guaranteed when it is popped from the priority queue and not when it is first discovered.

4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

   Use expanded because it measures number of states explored and it is less dependent on hardware speed. Runtime_ms however depends on the laptop's processor, system load and other factors.

5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?

   As such, GBFS does not account for the total cost incurred so far g(n) while A* accounts for both g(n) and the heuristic function h(n). That is the reason why GBFS might yield a solution that is much more expensive than the solutions found by using UCS or A*.

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?

   - UCS uses g(n). It expands Y first (3<8).
   - GBFS uses h(n). It expands X first (2<6).
   - A* uses f(n) = g(n) + h(n). f(n) = 9. IT expands Y first (9<10).

7. Why must *h* be expressed in the same units as the edge costs?

   A* calculates f(n) = g(n) + h(n). Both of them must be in compatible units, for example, kilometers, in order to provide meaningful results and to influence the search properly.

8. Lab2 computes $h\star$ by running Dijkstra on the reversed graph from the goal. Why reversed?

   Djikstra's algorithm finds the shortest path from a single source to all other nodes. Reversing the graph's edges and running it from the goal node, it will calculate the exact true cost from all nodes to the goal, producing the perfect heuristic h*.

9. Weighted A⋆ with *w = 1.5* changed the cost for KUL→KEF but not for SIN→EZE. What property of the SIN→EZE route explains this?

   If increasing the heuristic (w=1,5) did not change the cost or route for SIN to EZE, it indicates that the optimal path is highly constrained or strictly dominant. There were no competitive alternatives routes for the inflated heuristics to misguide the algorithm into exploring.